A Comparative Analysis of Memory Safety, Concurrency, and Performance in Edge-AI

Timothy Clarke
University of Dundee
United Kingdom
2712139@dundee.ac.uk

1. Introduction

1.1. Background and Context

In recent years, Edge-AI (Edge Artificial Intelligence) has begun to move the deployment of AI models from centralised cloud-based servers to local devices such as sensors, mobile phones, and embedded systems. This allows for real-time processing and reduces internet bandwidth usage, making it suitable for applications where reduced latency is critical (e.g. fitness trackers, autonomous vehicles, etc.), or where connectivity is unreliable or unavailable (e.g. remote weather stations, satellite image analysis, etc.).

TODO: discuss advances in hardware acceleration

Edge-AI deployment brings challenges in terms of resource constraints, such as limited computational power, memory, and power requirements. Remote software updates to maintain Edge-AI applications also come at a cost, as they can be expensive and time-consuming, especially when dealing with a large number of devices. These challenges make language selection an important design decision for Edge-AI pipelines.

136

1.2. Problem Statement

When deploying on Edge hardware, the above listed challenges amplify the impact of programming language choice. A language's runtime model dictates memory, concurrency, and scheduling behaviour under load, which directly impacts latency, throughput, and resource usage. For example, manual memory management offers fine-grained control and increased performance, but simultaneously increases the risk of memory leaks and undefined behaviour. Conversely, memory management may be automated through garbage collection (GC) at the cost of increased latency and unpredictable latency jitter.

Selecting a language for Edge-AI pipelines is often guided by familiarity or generalised benchmarks, rather than the evaluation of runtime models under stress with the constraints of embedded hardware. There is a lack of empirical evidence of how specific memory management and concurrency models interact with backpressure policies under heavy, fluctuating loads. Additionally, resource contention and thermal throttling confounders can introduce noise that limits the validity of naive comparisons.

This dissertation addresses this gap by providing empirical evidence and an evaluation of pipelines deployed on resource-constrained hardware, with a focus on the trade-offs among Rust, C++, and Python implementations of a dual-stream Human Activity Recognition (HAR) pipeline on industry-standard Edge-AI hardware. It focuses on three confounders: (1) language runtime models, (2) backpressure policies under various loads, and (3) thermal/power throttling.

209

1.3. Research Questions and Objectives

The primary research questions are:

RQ1 **Runtime Performance:** How do the runtime models of Rust, C++, and Python influence latency, throughput, and memory consumption in a dual-stream HAR pipeline on Edge-AI hardware?

RQ2 **Backpressure Interaction:** How do the language-specific runtime models interact with different backpressure policies under varying load, and what are the trade-offs in observed deadline adherence, system stability, and allocator/GC pressure and concurrency overhead?

RQ3 **Dynamic Profiling vs. Runtime Behaviour:** To what extent do dynamic memory/concurrency profiling metrics explain the observed performance bottlenecks and trade-offs under load?

The primary research objectives are:

RO1 Implement a functionally identical dual-stream HAR pipeline in Rust, C++, and Python, ensuring optimised idiomatic implementations for each language.

RO2 Evaluate the performance of each implementation under controlled conditions, using a shared deterministic load generator, to quantify how backpressure policies and runtime models impact latency, throughput, memory consumption, and thermal/power dynamics under load.

RO3 Analyse runtime model overhead to explain performance differences, and derive empirically grounded guidance for language selection in constrained Edge-AI deployments.

166

1.4. Research Contributions

This dissertation offers the following contributions to software engineering for Edge-AI systems:

C1 **Cross-Language Runtime Evaluation:** Addressing RQ1, this work delivers an empirical comparison of how Rust, C++, and Python runtime models (memory management and concurrency) influence latency, throughput, and memory consumption on resource-constrained embedded hardware.

C2 **Interaction Analysis:** Addressing RQ2, this work provides a controlled assessment of how backpressure policies interact with runtime models under various loads, and identifies trade-offs between throughput and long-term system stability.

C3 **Root-Cause Identification:** Addressing RQ3, this work presents empirical evidence linking dynamic memory allocation churn, GC pause duration, and scheduling overhead to system latency and throughput.

C4 **Evidence-Based Guidelines:** Addressing findings across all research questions, this dissertation offers empirically grounded recommendations for language selection in real-time, multi-stream edge deployments.

122

1.5. Scope and Limitations

The focus of this dissertation is on the interaction of three language runtime models (Rust, C++, and Python) with system latency and throughput, using standardised, idiomatic implementations of a dual-stream HAR pipeline on industry-standard Edge-AI hardware. The scope is limited to a standardised implementation representative of real-world multi-modal processing, with confounders limited to thermal/power throttling and queue-based backpressure policies.

The pipeline architecture, deterministic load generator, and backpressure mechanisms are designed for cross-platform compatibility.

However, AI acceleration, thermal behaviour, and power management are fundamentally SoC-dependent. Consequently, the profiling and acceleration tools used during evaluation (e.g., NVIDIA tegrastats, TensorRT) are specific to the Jetson Orin Nano platform and cannot be directly applied across other edge devices.

115

1.6. Dissertation Outline

The remainder of this dissertation is structured as follows: **Chapter 2** reviews related work, the runtime models of the target languages, and backpressure policies. **Chapter 3** details the methodology, including the hardware and software setup, backpressure interfaces, and profiling toolchains to collect metrics. **Chapter 4** describes the implementation of the HAR pipeline in each language, highlighting language-specific optimisations and challenges. **Chapter 5** presents the results, including performance metrics, memory allocation and GC pressure, and backpressure outcomes under varying loads. **Chapter 6** discusses the findings, and limitations of the study. Finally, **Chapter 7** concludes by addressing the research questions, providing practical recommendations for language selection in Edge-AI contexts, and suggesting directions for future work.

113

884

2. Literature Review

2

3. Methodology

3.1. Hardware and Software Environment

3.1.1. Hardware Stack

The NVIDIA Jetson Orin Nano Super was utilised as the target Edge-AI platform. It is a high-performance system-on-module (SoM) designed for Edge-AI development and allows complex AI workloads and multi-stream pipelines to run efficiently. The developer kit includes a carrier board with I/O interfaces (e.g., USB, Ethernet, DisplayPort), a MicroSD card slot for booting, and the Jetson Orin Nano module itself which includes the following features:

- 6-core Arm Cortex-A78AE 64-bit CPU for general-purpose concurrent processing
- up to 67 TOPS (Tera Operations Per Second) of AI performance
- 8 GB of 128-bit LPDDR5 memory with a bandwidth of 102 GB/s
- 1024 CUDA cores for general-purpose GPU computing
- 32 Tensor cores for AI acceleration

A Waveshare IMX219-160 Camera Module was used to deliver the RGB video stream, configured to capture at 1920×1080 RGB frames at 30 FPS, and connected via MIPI CSI-2 (Mobile Industry Processor Interface Camera Serial Interface 2). The images captured by the camera's 160° FOV required undistortion

The camera captures images with a field of view (FOV) of 160 degrees, making it suitable for capturing a wide area for human activity recognition.

A Bosch Sensortec BMI088 IMU Shuttle Board 3.0 was used to provide inertial measurement data, configured to capture 6-axis data, and connected via SPI for maximum throughput. The BMI088 combines a 3-axis accelerometer and a 3-axis gyroscope, providing two complementary data streams at up to 1.6 kHz (accelerometer) and 2.0 kHz (gyroscope).

To ensure that disk I/O did not cause bottlenecks or confound performance comparisons, all implementations were executed from a 1TB Samsung 990 PRO PCIe 4.0 NVMe M.2 SSD. A SanDisk "High-Endurance" microSD Card (64GB, Class 10/U3) was only used for initial device installation and bootloading, and was unmounted after boot to prevent any background I/O (such as writing logs) from interfering with performance measurements.

306

3.1.2. Software Stack

The Jetson was flashed with NVIDIA's JetPack 7.1 SDK, which includes Jetson Linux 38.4,and CUDA 13.0.0, cuDNN 9.12.0, and TensorRT 10.13.3.9 for AI acceleration. TODO: Mention L4T Ubuntu 24.04? What version of ONNX?

CUDA provides a parallel execution environment and programming model for NVIDIA GPUs, using Single Instruction Multiple Thread (SIMT) architecture. It allows developers to write a *kernel* function that is executed in parallel across many threads (using different data) on the GPU, enabling high-performance computing for AI workloads.

The kernel *threads* are grouped into *blocks* (up to 1024 threads per block), which in turn are grouped into a *grid*. Each block is split into *warps* of 32 threads that are executed simultaneously on a single GPU *Streaming Multiprocessor* (SM). Developers must explicitly manage the transfer of data between the host (CPU) and device (GPU).

cuDNN (CUDA Deep Neural Network) is a GPU-accelerated library that sits on top of CUDA and runs on the GPU to provide higher-level abstractions and optimised implementations of common deep learning operations (e.g., normalisation, transformation, softmax, etc.). TODO: How to ensure all pipeline implementations route through an identical cuDNN-backed execution path, if cuDNN uses a heuristic search to select the fastest implementation?

TensorRT is responsible for optimising and running the AI models on the GPU from a *.engine* file generated from an **ONNX** (Open Neural Network Exchange) model.

Performance and overhead of the language runtime models was measured at the boundary of the host/device (CPU/GPU) memory separation, where the CPU manages the runtime model and the SIMT architecture runs the AI workloads on the GPU. This prevents confounding the results with hardware latency, and provides a clearer comparison of how each language's runtime model performs under load and backpressure.

289

602

4. Implementation

5. Results

6. Discussion

7. Conclusion

Total words: 1517